

labsheet =04 computer vision Priyanshu Sharma Cu240250980 section=(B) (47)

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
```

```
# FREQUENCY DOMAIN IMAGE FILTERING USING FOURIER TRANSFORM
# =====

print("=" * 65)
print("FREQUENCY DOMAIN IMAGE FILTERING USING FOURIER TRANSFORM")
print("=" * 65)

# -----
# 1. LOAD IMAGE
# -----

image_path = "/content/pexels-friededia-30649280.jpg"
image = cv2.imread(image_path)

if image is None:
    print("ERROR: Image not found!")
    print("Check your image path.")
    exit()

print("\nOriginal image loaded.")
print("Original image shape:", image.shape)
```

```
=====
FREQUENCY DOMAIN IMAGE FILTERING USING FOURIER TRANSFORM
=====

Original image loaded.
Original image shape: (5464, 8192, 3)
```

```
# 2. CONVERT TO GRAYSCALE
# -----

image = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)

print("After grayscale conversion:", image.shape)

# -----
# 3. RESIZE IMAGE
# -----

image = cv2.resize(image, (800, 600))

print("After resizing:", image.shape)
```

```
After grayscale conversion: (5464, 8192)
After resizing: (600, 800)
```

```
# 4. CONVERT IMAGE TO FLOAT32
# -----

image_float = image.astype(np.float32)

print("Image data type:", image_float.dtype)
```

```
Image data type: float32
```

```
# 5. COMPUTE FOURIER TRANSFORM
# -----

dft = cv2.dft(
    image_float,
    flags=cv2.DFT_COMPLEX_OUTPUT
)

print("\nDFT calculated successfully.")
```

```
DFT calculated successfully.
```

```
# 6. SHIFT ZERO FREQUENCY TO CENTER
```

```
# -----
```

```
dft_shift = np.fft.fftshift(dft)
```

```
# 7. MAGNITUDE SPECTRUM
```

```
# -----
```

```
magnitude_spectrum = cv2.magnitude(  
    dft_shift[:, :, 0],  
    dft_shift[:, :, 1]  
)
```

```
magnitude_spectrum = 20 * np.log(  
    magnitude_spectrum + 1  
)
```

```
magnitude_spectrum = cv2.normalize(  
    magnitude_spectrum,  
    None,  
    0,  
    255,  
    cv2.NORM_MINMAX  
)
```

```
magnitude_spectrum = np.uint8(  
    magnitude_spectrum  
)
```

```
print("Frequency spectrum calculated.")
```

```
Frequency spectrum calculated.
```

```
# 8. IMAGE CENTER
```

```
# -----
```

```
rows, cols = image.shape
```

```
crow = rows // 2
```

```
ccol = cols // 2
```

```
radius = 40
```

```
# 9. LOW-PASS FILTER
```

```
# -----
```

```
low_pass_mask = np.zeros(  
    (rows, cols, 2),  
    np.float32  
)
```

```
cv2.circle(  
    low_pass_mask,  
    (ccol, crow),  
    radius,  
    (1, 1),  
    -1  
)
```

```
# Apply filter
```

```
low_pass_dft = dft_shift * low_pass_mask
```

```
# Shift back
```

```
low_pass_ishift = np.fft.ifftshift(  
    low_pass_dft  
)
```

```
# Inverse Fourier Transform
```

```
low_pass_result = cv2.idft(  
    low_pass_ishift  
)
```

```
# Magnitude
```

```
low_pass_result = cv2.magnitude(  
    low_pass_result[:, :, 0],  
    low_pass_result[:, :, 1]  
)
```

```
# Normalize
low_pass_result = cv2.normalize(
    low_pass_result,
    None,
    0,
    255,
    cv2.NORM_MINMAX
)

low_pass_result = np.uint8(
    low_pass_result
)

print("Low-pass filter completed.")
```

Low-pass filter completed.

```
# 10. HIGH-PASS FILTER
# -----

high_pass_mask = np.ones(
    (rows, cols, 2),
    np.float32
)

cv2.circle(
    high_pass_mask,
    (ccol, crow),
    radius,
    (0, 0),
    -1
)

# Apply filter
high_pass_dft = dft_shift * high_pass_mask

# Shift back
high_pass_ishift = np.fft.ifftshift(
    high_pass_dft
)

# Inverse Fourier Transform
high_pass_result = cv2.idft(
    high_pass_ishift
)

# Magnitude
high_pass_result = cv2.magnitude(
    high_pass_result[:, :, 0],
    high_pass_result[:, :, 1]
)

# Normalize
high_pass_result = cv2.normalize(
    high_pass_result,
    None,
    0,
    255,
    cv2.NORM_MINMAX
)

high_pass_result = np.uint8(
    high_pass_result
)

print("High-pass filter completed.")
```

High-pass filter completed.

```
# 11. SAVE OUTPUTS
# -----

cv2.imwrite(
    "original_image.jpg",
    image
)

cv2.imwrite(
    "frequency_spectrum.jpg",
    magnitude_spectrum
)
```

```
)

cv2.imwrite(
    "low_pass_filtered.jpg",
    low_pass_result
)

cv2.imwrite(
    "high_pass_filtered.jpg",
    high_pass_result
)

print("\nAll output images saved successfully.")
```

All output images saved successfully.

```
# 12. DISPLAY RESULTS
# -----

plt.figure(figsize=(12, 8))

plt.subplot(2, 2, 1)
plt.imshow(image, cmap="gray")
plt.title("Original Image")
plt.axis("off")

plt.subplot(2, 2, 2)
plt.imshow(magnitude_spectrum, cmap="gray")
plt.title("Frequency Spectrum")
plt.axis("off")

plt.subplot(2, 2, 3)
plt.imshow(low_pass_result, cmap="gray")
plt.title("Low-Pass Filtered Image")
plt.axis("off")

plt.subplot(2, 2, 4)
plt.imshow(high_pass_result, cmap="gray")
plt.title("High-Pass Filtered Image")
plt.axis("off")

plt.tight_layout()

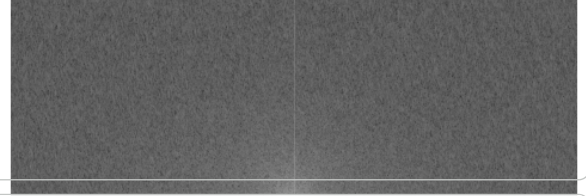
plt.savefig(
    "frequency_filtering_result.jpg",
    dpi=300
)

plt.show()
```

Original Image



Frequency Spectrum



```
# 13. FINAL OUTPUT
```

```
# -----
```

```
print("\n" + "=" * 65)
print("EXPERIMENT COMPLETED SUCCESSFULLY")
print("=" * 65)
```

```
print("\nGenerated files:")
print("1. original_image.jpg")
print("2. frequency_spectrum.jpg")
print("3. low_pass_filtered.jpg")
print("4. high_pass_filtered.jpg")
print("5. frequency_filtering_result.jpg")
print("PRIYANSHU SHARMA CU240250980 ")
```

```
=====
EXPERIMENT COMPLETED SUCCESSFULLY
=====
```

```
Generated files:
1. original_image.jpg
2. frequency_spectrum.jpg
3. low_pass_filtered.jpg
4. high_pass_filtered.jpg
5. frequency_filtering_result.jpg
PRIYANSHU SHARMA CU240250980
```

